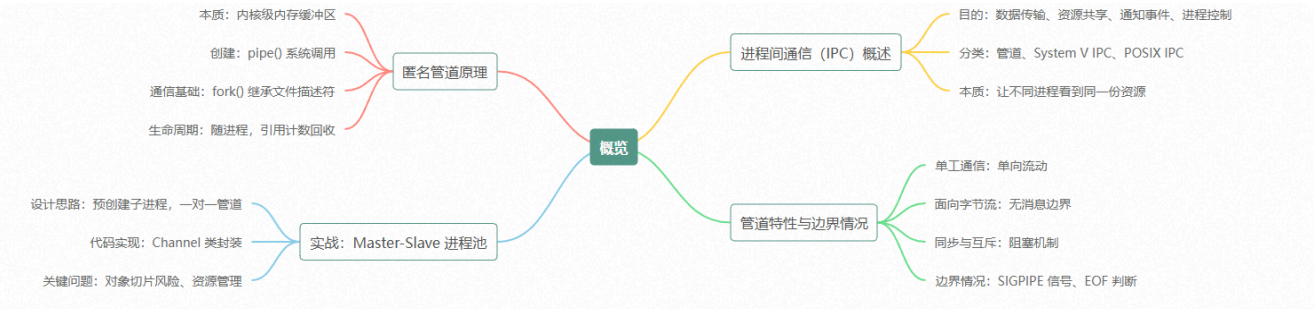
2026-7-27 Linux进程--通信概述

- 1. 进程间通信概述
  - 1.1 为什么需要进程间通信
  - 1.2 进程间通信的四类目的
  - 1.3 IPC 的本质问题
  - 1.4 IPC 的发展与分类
- 2. 管道与匿名管道
  - 2.1 管道是什么
  - 2.2 为什么匿名管道能够让父子进程通信
  - 2.3 pipe() 系统调用
  - 2.4 为什么要在父子进程中关闭不用的端点
  - 2.5 最小通信示例
  - 2.6 从内核角度看管道
- 3. 管道的读写语义与边界情况
  - 3.1 管道是面向字节流的
  - 3.2 阻塞与非阻塞读写
  - 3.3 四种典型边界情况
  - 3.4 匿名管道的五个特点
  - 3.5 常见错误
- 4. IPC 实战：Master-Slave 进程池
  - 4.1 为什么使用进程池
  - 4.2 信道模型
  - 4.3 文件描述符继承与资源边界
  - 4.4 Channel 的封装与对象复制
  - 4.5 进程池核心代码骨架
  - 4.6 进程池设计要点

1. 进程间通信概述



1.1 为什么需要进程间通信

进程具有独立性：每个进程拥有自己的虚拟地址空间，一个进程通常不能直接读写另一个进程中的变量。

`fork()` 之后，父子进程虽然最初看到相似的内存内容，但物理内存采用写时拷贝：任一方修改数据时，内核才复制相应页面。因此，普通变量无法承担两个进程之间持续、双向的数据传递。

进程间通信（Inter-Process Communication，IPC）研究的正是：**彼此独立的进程如何协同工作、交换数据并控制执行顺序。**

1.2 进程间通信的四类目的

1. **数据传输**：一个进程把数据发送给另一个进程。
2. **资源共享**：多个进程共享同一份资源。
3. **通知事件**：一个进程通知另一个或一组进程，某项事件已经发生，例如子进程退出。
4. **进程控制**：一个进程控制另一个进程，例如调试器拦截目标进程的异常并改变其执行状态。

1.3 IPC 的本质问题

进程间通信首先遇到的不是“怎样调用函数”，而是下面两个问题：

1. 进程彼此独立，不能直接访问对方的地址空间。
2. 通信双方必须先看到同一份资源。

这份共享资源可以由文件、内核缓冲区、共享内存或消息对象承载，但都必须由操作系统参与建立和管理。用户程序通过系统调用申请资源，再通过 `read()`、`write()`、发送消息等接口访问它。

> **本质：让不同的进程看到同一份资源，并由操作系统保证访问方式和管理边界。**

两个进程看到的是自己的虚拟地址，并不要求使用完全相同的虚拟地址；关键是它们最终指向同一个底层对象。

1.4 IPC 的发展与分类

课堂按照三个阶段梳理 IPC：

1. 管道：Unix 中最古老的通信形式之一。
2. System V IPC：本机通信中较早的一套标准。
3. POSIX IPC：更现代的一组接口，课堂把它视为进一步面向网络与通用通信场景的发展。

严格区分时，真正跨主机通信常用套接字；POSIX IPC 仍然主要服务于同一台机器上的进程。下面的分类用于建立知识地图：

类别	包含的机制	主要特征
管道	匿名管道 `pipe`、命名管道 `FIFO`	面向字节流；匿名管道依赖继承关系
System V IPC	消息队列、共享内存、信号量	一组较早的本机 IPC 标准
POSIX IPC	消息队列、共享内存、信号量、互斥量、条件变量、读写锁	接口更现代，部分机制兼顾同步

本篇只沿板书的行文继续展开“匿名管道及其在进程池中的应用”；命名管道、共享内存、消息队列与信号量只保留在分类中，不在本篇提前展开。

2. 管道与匿名管道

2.1 管道是什么

管道把一个进程产生的数据流连接到另一个进程。应用看到的是两个文件描述符，内核则在两者之间维护一块通信缓冲区。

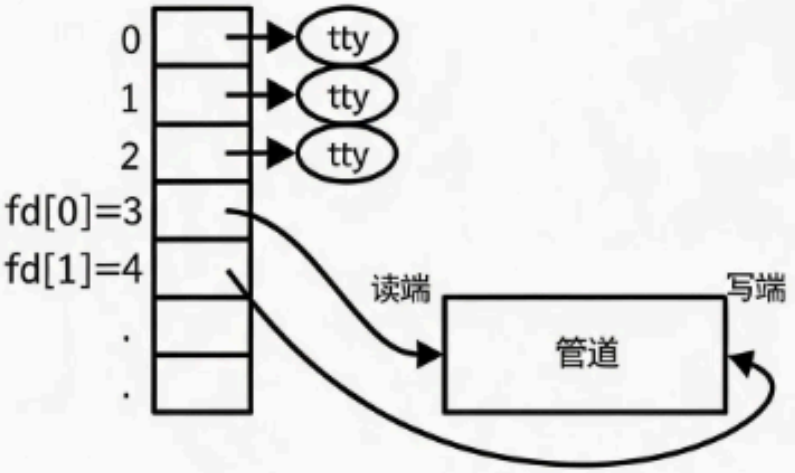
匿名管道有三个关键特征：

1. 没有用户可见的文件路径和文件名。
2. 只存在于进程打开的文件描述符与内核对象中。
3. 通常只能传给具有继承关系的子进程，例如通过 `fork()` 共享。

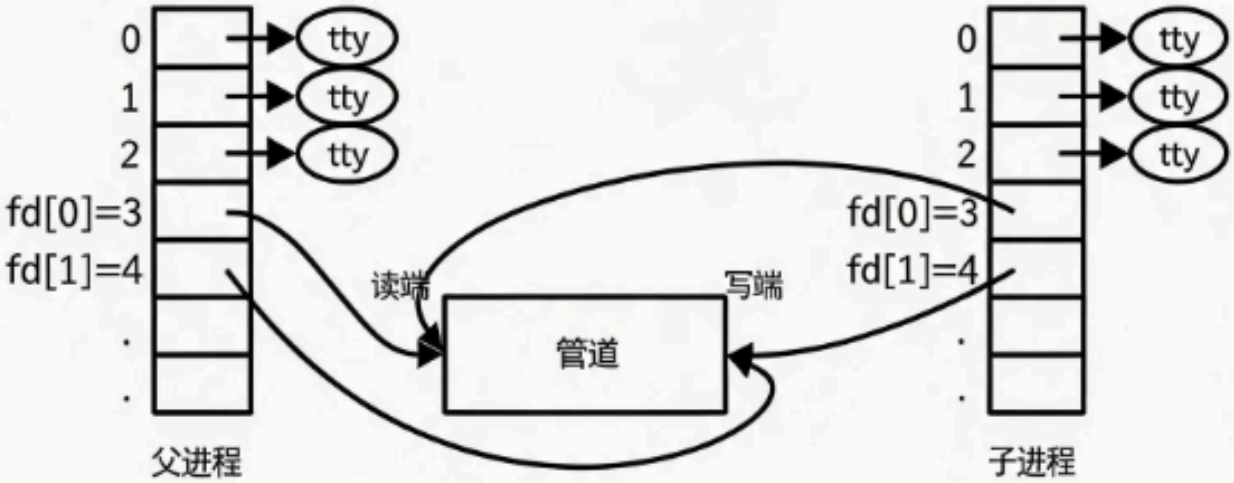
2.2 为什么匿名管道能够让父子进程通信

匿名管道仿照文件建立了“读端 + 写端”的通信模型，但不需要在磁盘上创建普通文件。

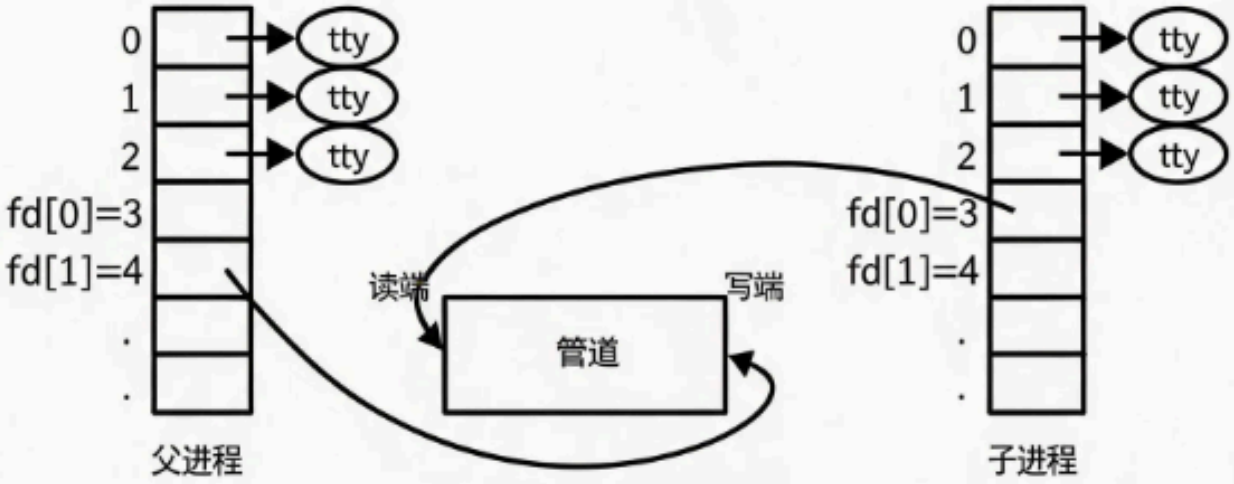
1. 父进程创建管道



2. 父进程fork出子进程



3. 父进程关闭fd[0]，子进程关闭fd[1]



父子进程能够通信，关键在文件描述符的继承：

1. 父进程调用 `pipe()`，得到两个文件描述符，分别指向同一个内核管道对象的读端和写端。
2. 父进程调用 `fork()`。子进程复制父进程的文件描述符表，因此也拿到了这两个描述符。
3. 两份文件描述符表里的表项最终指向同一个打开文件描述和同一个内核管道缓冲区。
4. 一方写入管道，另一方可以通过另一个端点读取。

这里复制的重点是**文件描述符表中的引用**，而不是深层复制 `struct file`、文件操作表或管道缓冲区。内核用引用计数管理这些共享对象，直到最后一个引用关闭才回收。

因为子进程继承了同一个内核对象的引用，所以匿名管道不需要路径名也能完成通信。它在使用方式上像文件，在生命周期上却与打开它的进程和描述符有关。

2.3 `pipe()` 系统调用

```
NAME
       pipe, pipe2 - create pipe

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int pipe(int pipefd[2]);

       #define _GNU_SOURCE             /* See feature_test_macros(7) */
       #include <fcntl.h>              /* Definition of O_* constants */
       #include <unistd.h>

       int pipe2(int pipefd[2], int flags);

       /* On Alpha, IA-64, MIPS, SuperH, and SPARC/SPARC64, pipe() has the
          following prototype; see VERSIONS */

       #include <unistd.h>

       struct fd_pair {
           long fd[2];
       };
       struct fd_pair pipe(void);
```

```c
#include <unistd.h>

int pipe(int pipefd[2]);
```

`pipefd` 是输出型参数：

- `pipefd[0]`：读端。
- `pipefd[1]`：写端。
- 成功返回 `0`。
- 失败返回 `-1`，并设置 `errno`。

`pipe()` 同时创建了一个单向通信通道，并以“一个读端、一个写端”的形式返回两个文件描述符。之后就可以像访问普通文件一样对它们调用 `read()` 和 `write()`。

> Linux 还提供 `pipe2(int pipefd[2], int flags)`，可以在创建时设置 `O_CLOEXEC`、`O_NONBLOCK` 等标志。

2.4 为什么要在父子进程中关闭不用的端点

`fork()` 之后，父子进程默认都持有读端和写端。若只进行单向通信，就应让每个进程只保留自己需要的方向：

- 父进程写、子进程读：父进程关闭读端，子进程关闭写端。
- 子进程写、父进程读：子进程关闭读端，父进程关闭写端。

不关闭也能运行，但容易误读写，还会影响边界判断。例如，只要系统中仍有任意一个写端没有关闭，读端就不能把 `read() == 0` 视为“所有写者都结束了”。

管道本身没有固定的“父进程必须写、子进程必须读”规定，方向由双方如何使用两个端点决定。

2.5 最小通信示例

下面的程序让父进程写、子进程读：

```
#include <cstring>
#include <iostream>
#include <sys/wait.h>
#include <unistd.h>

int main()
{
    int pipefd[2];
    if (pipe(pipefd) == -1)
    {
```

```
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();
    if (pid == -1)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        close(pipefd[1]); // 子进程只读

        char buffer[128];
        ssize_t n = read(pipefd[0], buffer, sizeof(buffer) - 1);
        if (n > 0)
        {
            buffer[n] = '\0';
            std::cout << "child received: " << buffer << std::endl;
        }

        close(pipefd[0]);
        return 0;
    }

    close(pipefd[0]); // 父进程只写

    const char *message = "Hello from parent";
    write(pipefd[1], message, std::strlen(message));

    close(pipefd[1]); // 写完后关闭，子进程最终读到 EOF
    waitpid(pid, nullptr, 0);
    return 0;
}
```

实际工程中还应处理 `write()` 的返回值与短写，并根据业务选择合适的协议；这里只保留最小通信骨架。

2.6 从内核角度看管道

管道能像文件一样使用，是因为内核为端点提供了与文件一致的操作接口：

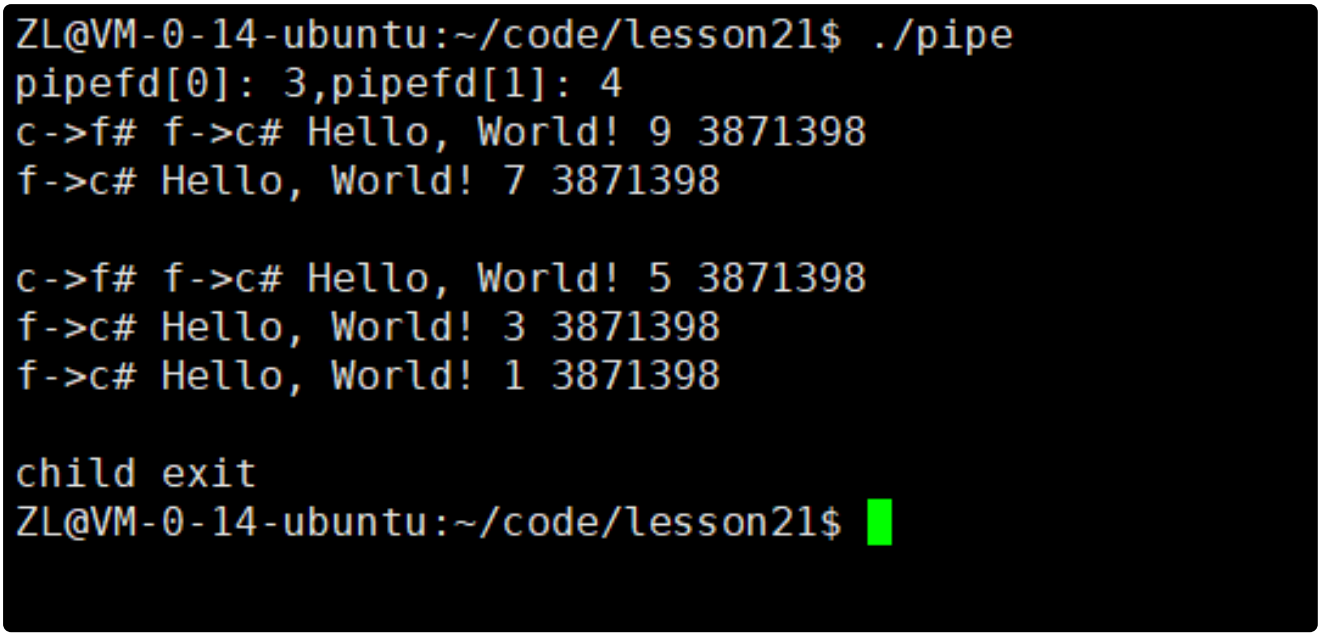
```
进程 -> 文件描述符 -> struct file -> 管道对象 -> 内核缓冲区
```

因此，读写管道与读写文件在接口层面高度一致，这体现了 Linux 的“一切皆文件”思想。区别在于：

- 普通文件的持久数据主要位于外部存储。
- 匿名的管道缓冲区位于内核内存中。
- 管道没有用户可打开的文件路径。

3. 管道的读写语义与边界情况

3.1 管道是面向字节流的



管道不保存写入次数，也不保留“消息边界”。写入方多次写入的数据，读取方可能一次读出；一次写入的数据，也可能分多次读出。

因此：

- 不能把一次 `write()` 默认等同于一条完整消息。
- 若要传输结构化消息，应额外约定消息头、长度字段或分隔符。
- `read()` 返回的是本次实际读到的字节数，不表示写入方的调用次数。

3.2 阻塞与非阻塞读写

场景	`O_NONBLOCK` 关闭	`O_NONBLOCK` 打开
管道为空时读取	`read()` 阻塞，直到有数据或所有写端关闭	立即返回 `-1`，`errno = EAGAIN`
管道已满时写入	`write()` 阻塞，直到读者腾出空间或读端关闭	立即返回 `-1`，`errno = EAGAIN`
所有写端关闭后读取	读走剩余数据后返回 `0`，表示 EOF	读走剩余数据后返回 `0`
所有读端关闭后写入	触发 `SIGPIPE`	同样触发 `SIGPIPE`；若忽略该信号，则写入返回 `-1`、`errno = EPIPE`

当一次写入的数据量不超过 `PIPE_BUF` 时，Linux 保证这次写入具有原子性；超过 `PIPE_BUF` 时，不保证多个写入进程的数据不会交错。

3.3 四种典型边界情况

1. **管道为空，读端继续读**：读端阻塞，等待写端提供数据。
2. **管道已满，写端继续写**：写端阻塞，等待读端取走数据。
3. **所有写端关闭，读端继续读**：读端先读完缓冲区中的剩余数据，再得到 `read() == 0`。
4. **所有读端关闭，写端继续写**：内核向写进程发送 `SIGPIPE`。默认动作是终止写进程。

```
ZL@VM-0-14-ubuntu:~/code/lesson21$ ./pipe
pipefd[0]: 3,pipefd[1]: 4
c->f# f->c# Hello, World! 5 3931403

child quit code: 0 signal: 13
```

截图显示的是 `waitpid` 观察到的状态：退出码为 `0`，终止信号为 `13`，即 `SIGPIPE`。在 shell 中看到的进程状态通常会写成 `141`，即 `128 + 13`；这两个数字表达的是同一事件，不能混称为“进程主动退出码 13”。

如果进程希望自行处理这种错误，可以忽略 `SIGPIPE`，再检查 `write()` 是否返回 `-1` 且 `errno == EPIPE`。

3.4 匿名管道的五个特点

1. **有亲缘关系**：普通匿名管道依赖文件描述符继承，只适用于共同祖先派生出的进程。

2. **单向通信**：一个管道只能稳定地承担一个方向；双向通信需要两个管道。
3. **面向字节流**：不保留消息边界。
4. **生命周期与引用有关**：所有相关描述符都关闭后，内核才回收管道。
5. **自带同步与互斥**：读空会阻塞、写满会阻塞，底层操作会进行协调；`PIPE_BUF` 以内的小块写入还具有原子性。

3.5 常见错误

- 通信结束后不关闭多余的端点，导致另一端无法收到 EOF。
- 把一次 `write()` 当作一条消息，忽略字节流语义。
- 只关闭一端就假设管道已经彻底销毁，忽略其他进程仍持有端点的情况。
- 不处理 `SIGPIPE`，读端意外退出后写进程也被默认终止。
- 认为父子进程拥有各自的管道对象；实际上它们分别持有指向同一内核对象的端点。

4. IPC 实战：Master-Slave 进程池

4.1 为什么使用进程池

频繁地 `fork()` 会带来创建进程、准备资源与回收子进程的开销。进程池采用“提前创建、循环复用”的思路：

1. 父进程预先创建若干个 Slave 子进程。
2. 每个子进程持有一个独立的读端。
3. Master 父进程保存每个子进程的 pid 和对应写端。
4. 有任务时，Master 通过管道把任务码发送给某个 Slave。
5. 任务结束后，关闭所有写端，让子进程读到 EOF，再统一 `waitpid()` 回收。

一个管道对应一个子进程，可以形成清晰的一对一信道，避免多个子进程同时从同一管道抢任务时产生难以追踪的竞争。

4.2 信道模型

Master 需要管理两个核心信息：

- 子进程的 pid，用于等待和回收。
- 写端文件描述符，用于派发任务。

子进程则只保留自己的读端，在循环中执行：

```
read(task_code)
    ├─ 读到完整任务码 -> 执行任务 -> 继续等待
    ├─ 读到 0          -> 所有写端已关闭，退出
    └─ 读取出错        -> 记录错误并退出
```

所有进程对任务码的宽度必须一致。例如统一使用 4 字节整数；若任务本身携带变长参数，还需要设计消息头和长度字段。

4.3 文件描述符继承与资源边界

每创建一个新子进程，它都会继承此前已经建立的管道描述符；同时还会继承当前管道的两端。正确的关闭顺序是：

1. 子进程关闭父进程拥有的历史写端。
2. 子进程关闭当前管道的写端，只保留当前读端。
3. 父进程关闭当前管道的读端，只保留当前写端。

```
ZL@VM-0-14-ubuntu:~$ ls /proc/463356/fd -l
total 0
lrwx------ 1 ZL ZL 64 Jul 30 18:22 0 -> /dev/pts/6
lrwx------ 1 ZL ZL 64 Jul 30 18:22 1 -> /dev/pts/6
lrwx------ 1 ZL ZL 64 Jul 30 18:22 2 -> /dev/pts/6
lr-x------ 1 ZL ZL 64 Jul 30 18:22 3 -> 'pipe:[45906585]'
ZL@VM-0-14-ubuntu:~$ ls /proc/463358/fd -l
total 0
lrwx------ 1 ZL ZL 64 Jul 30 18:23 0 -> /dev/pts/6
lrwx------ 1 ZL ZL 64 Jul 30 18:23 1 -> /dev/pts/6
lrwx------ 1 ZL ZL 64 Jul 30 18:23 2 -> /dev/pts/6
lr-x------ 1 ZL ZL 64 Jul 30 18:23 3 -> 'pipe:[45906586]'
l-wx------ 1 ZL ZL 64 Jul 30 18:23 4 -> 'pipe:[45906585]'
ZL@VM-0-14-ubuntu:~$ ls /proc/463359/fd -l
total 0
lrwx------ 1 ZL ZL 64 Jul 30 18:23 0 -> /dev/pts/6
lrwx------ 1 ZL ZL 64 Jul 30 18:23 1 -> /dev/pts/6
lrwx------ 1 ZL ZL 64 Jul 30 18:23 2 -> /dev/pts/6
lr-x------ 1 ZL ZL 64 Jul 30 18:23 3 -> 'pipe:[45906587]'
l-wx------ 1 ZL ZL 64 Jul 30 18:23 4 -> 'pipe:[45906585]'
l-wx------ 1 ZL ZL 64 Jul 30 18:23 5 -> 'pipe:[45906586]'
ZL@VM-0-14-ubuntu:~$ ls /proc/463360/fd -l
total 0
lrwx------ 1 ZL ZL 64 Jul 30 18:23 0 -> /dev/pts/6
lrwx------ 1 ZL ZL 64 Jul 30 18:23 1 -> /dev/pts/6
lrwx------ 1 ZL ZL 64 Jul 30 18:23 2 -> /dev/pts/6
lr-x------ 1 ZL ZL 64 Jul 30 18:23 3 -> 'pipe:[45906593]'
l-wx------ 1 ZL ZL 64 Jul 30 18:23 4 -> 'pipe:[45906585]'
l-wx------ 1 ZL ZL 64 Jul 30 18:23 5 -> 'pipe:[45906586]'
l-wx------ 1 ZL ZL 64 Jul 30 18:23 6 -> 'pipe:[45906587]'
ZL@VM-0-14-ubuntu:~$
```

截图可以看到，子进程除了自己的管道端点，还继承了此前创建的写端。如果不主动关闭这些历史写端，管道可能永远无法达到“所有写端都已关闭”的状态，子进程也就等不到 EOF。

4.4 Channel 的封装与对象复制

`Channel` 通常封装 `write_fd`、`child_pid` 和信道名称。它既保存系统资源，也保存资源的所有权，因此不能让多个对象误以为自己都可以关闭同一个 fd。

```
创建子进程：461735成功...
创建子进程：461750成功...
创建子进程：461757成功...
创建子进程：461767成功...
创建子进程：461793成功...
wfd=4,who= 461735,channel name=sub-channel-461735
wfd=4,who= 461735,channel name=sub-channel-461735
wfd=5,who= 461750,channel name=sub-channel-461750
wfd=5,who= 461750,channel name=sub-channel-461750
wfd=6,who= 461757,channel name=sub-channel-461757
wfd=6,who= 461757,channel name=sub-channel-461757
wfd=7,who= 461767,channel name=sub-channel-461767
wfd=7,who= 461767,channel name=sub-channel-461767
wfd=8,who= 461793,channel name=sub-channel-461793
wfd=8,who= 461793,channel name=sub-channel-461793
```

截图中的输出出现了重复的信道与 fd 信息，说明对象在复制过程中形成了重复引用。此时即使并不立即报错，也可能在销毁、关闭或回收时产生重复关闭和归属不清的问题。

可以采用两种方式避免：

- 把 `Channel` 设为只移动、禁止复制，并正确实现资源转移。
- 在容器中保存 `std::unique_ptr<Channel>` 等拥有明确唯一所有者的对象。

4.5 进程池核心代码骨架

下面的代码只保留最关键的创建、派发与回收逻辑：

```cpp
#include <cerrno>
#include <csignal>
#include <cstdint>
#include <iostream>
#include <string>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <vector>

struct Channel
{
    int write_fd;
```

```
    pid_t child_pid;
};

static void Worker(int read_fd)
{
    for (;;)
    {
        std::int32_t task_code = -1;
        ssize_t n = read(read_fd, &task_code, sizeof(task_code));

        if (n == static_cast<ssize_t>(sizeof(task_code)))
        {
            std::cout << "worker " << getpid()
                      << " run task " << task_code << '\n';
        }
        else if (n == 0)
        {
            break; // Master 已关闭所有写端
        }
        else if (n < 0 && errno == EINTR)
        {
            continue;
        }
        else
        {
            perror("read");
            break;
        }
    }
}

int main()
{
    signal(SIGPIPE, SIG_IGN);
    std::vector<Channel> channels;

    const int worker_count = 4;
    for (int i = 0; i < worker_count; ++i)
    {
        int pipefd[2];
        if (pipe(pipefd) == -1)
        {
            perror("pipe");
            return 1;
        }
```

```
        pid_t pid = fork();
        if (pid == -1)
        {
            perror("fork");
            return 1;
        }

        if (pid == 0)
        {
            // 子进程关闭此前创建的所有写端
            for (const auto &channel : channels)
                close(channel.write_fd);

            close(pipefd[1]);
            Worker(pipefd[0]);
            close(pipefd[0]);
            _exit(0);
        }

        close(pipefd[0]);
        channels.push_back({pipefd[1], pid});
    }

    // 轮询派发 8 个任务
    for (int task = 0; task < 8; ++task)
    {
        Channel &channel = channels[task % channels.size()];
        std::int32_t code = task;
        ssize_t n = write(channel.write_fd, &code, sizeof(code));

        if (n == -1 && errno == EPIPE)
            break;
    }

    // 先关闭所有写端，让子进程读到 EOF
    for (auto &channel : channels)
    {
        close(channel.write_fd);
        channel.write_fd = -1;
    }

    // 再统一回收所有子进程
    for (auto &channel : channels)
        waitpid(channel.child_pid, nullptr, 0);
```

```
    return 0;
}
```

如果 Master 先关闭写端，再逐个回收子进程是可以的；若采用“关闭一个就等待对应子进程”的方式，也应注意其他仍然存活的管道不会影响这个子进程的 EOF 判断。关键不是某一种固定写法，而是保证**每个子进程只依赖自己的信道结束，所有写端最终都能关闭，所有子进程都能被回收**。

4.6 进程池设计要点

1. **先约定协议**：任务码宽度固定，复杂任务要加入长度和类型。
2. **保持一一对应**：每个 Slave 一条管道，Master 保存对应 pid 和写端。
3. **彻底关闭继承端点**：子进程关闭历史写端和当前写端，Master 关闭当前读端。
4. **用 EOF 表示结束**：Master 关闭全部写端后，Worker 的 `read()` 最终返回 `0`。
5. **明确资源所有权**：fd 与 pid 应只有一个管理对象，对象复制需遵守移动语义。
6. **统一回收**：关闭通信端点后调用 `waitpid()`，避免僵尸进程。
7. **处理异常写端**：忽略 `SIGPIPE` 并检查 `EPIPE`，防止一个 Worker 异常退出拖垮整个 Master。

> **内容命名建议：**《Linux 进程间通信：从匿名管道到 Master-Slave 进程池》